[EXECUTIVE TECH WHITEPAPER · 68 INSTRUCTOR SLIDES + REPO SOURCE INTEGRATED]

AI 에이전트를 위한 2nd Brain과 Eval 프레임워크 아키텍처

분석 대상: 메타 시니어 엔지니어 하재상(실밸개발자) 77분 세미나 + 강사 원본 슬라이드 68장 + 공개 레포(jha0313/second-brain-eval-kit) 전수 분석
적용 분야: 모바일 통신/물리계층(PHY) 임베디드 C/C++ SW 코드 리뷰 및 자율 검증 파이프라인

[핵심 요약 (Executive Summary)]
본 백서는 메타 시니어 엔지니어 하재상님의 강연 보이스, CV로 캡처한 원본 슬라이드 68장, 그리고 오픈소스 레포지토리(github.com/jha0313/second-brain-eval-kit)의 실제 구현 코드를 통합 분석한 엔터프라이즈 기술 백서이다. 자율 코딩 에이전트 도입의 성패를 가르는 두 기둥인 1) 컨텍스트 계층(codebase-brain: TS/Clang AST 그래프 query.mjs + SQLite FTS5 BM25/RRF 코드 메모리 why.mjs + 3열 라우팅 테이블)과 2) 검증 계층(skill-eval: 4대 무균 격리 isolation.py + 0점의 3가지 이름 checks.py + K=3 다수결 Judge + 95% 신뢰구간 판독)을 규명하고, 모바일 통신 물리계층(PHY) 임베디드 SW 그룹의 아키텍처로 1:1 매핑한다.

1. 검증 병목으로의 이동 & AutoKliq 프로덕션 코드베이스 실전 실험 (Slide #13)

LLM 도입 이후 코드 작성 속도는 폭발했으나, 초대형 PR 범람으로 인해 '인간 리뷰어의 검증 한계'가 새로운 병목이 되었다. 메타 사내에서 월 $30,000 상당의 토큰 실험과 성능 프로파일링 트리아지 단축(10시간 -> 30분, Profiling MCP + 시니어 스킬 + Brain 결합)을 거치며 입증된 AutoKliq(6,230개 파일) 실전 파급 반경 실험 결과는 다음과 같다:

비교 항목 (질문: lib/credit-reservations.ts 수정 시 파급 파일 전수 탐색)	스킬 없이 (Without Skill · Baseline)	codebase-brain 스킬 적용 (With Skill)
에이전트 행동 패턴	Grep 2회 실패 후, 매번 일회용 import 역추적 스크립트를 즉석에서 새로 작성(3/3)하여 실행	brain build(12.19초, 56.8MB DB) 후 brain impact 명령 단 한 줄로 9-Hop BFS 즉시 추적
소요 시간 및 탐색 결과	56초 (394 / 394개 파일 발견, 매번 스크립트 재발명)	< 1초 (394개 파일 + 144개 테스트 100% 결정론적 반환: 1-hop 3 -> 2-hop 21 -> 3-hop 146 -> 4-hop+ 224)

2. 개발자용 2nd Brain (`codebase-brain`)의 3대 핵심 구성요소 (Slides #16~#27)

Tiago Forte(2022)의 일반 세컨드 브레인은 소비자가 '사람'이고 PARA 폴더에 노트를 정리하지만, 개발자용 세컨드 브레인은 소비자가 '코딩 에이전트'이며 핵심 원칙은 "낡은 컨텍스트(Stale Context)는 없는 것보다 나쁘다"이다.

구성요소	구현 파일 및 핵심 알고리즘	환각 및 노이즈 방지 메커니즘 (강사 슬라이드 핵심)
① 지도 (Map)	src/query.mjs TS Compiler API AST 파싱, 고유 Symbol ID(path#SymbolName), N-Hop BFS 역추적	3대 커버리지 라벨: complete(완전 탐색), truncated(500개/10홉 상한 도달), partial(동적 호출/미해결 엣지 존재) + stale(커밋 불일치 경고)
② 코드 메모리	src/why.mjs SQLite FTS5 bm25(chunks_fts, 3.0, 1.0) + RRF(1/(60+rank+1)) 하이브리드 랭킹	"코드는 저장하지 않고 지도에 잇는다" — LLM 비용 $0, <1초에 문서/커밋(b8401c2a) 인용. 함수 이름은 후보 1~3개일 때만 연결(헷갈리면 잇지 않는다)
③ 라우팅	SKILL.md 3열 테이블: The question | Call | Not this	작은 단일 파일 수정에는 "do not call brain. Read the file." 명시 -> 평가 결과 필요 작업 발동률 76.8%(53/69), 작은 작업 오발동 0/6(0%) 달성

3. Skill Eval (`skill-eval`): 2026년의 숫자 3개 & pass@3 vs pass^3 (Slides #34~#43)

• 2026년의 숫자 3개 (#34): ① 56%(스킬을 줬지만 절반 이상 안 불림 — Vercel Next.js 16, 2026.01), ② 6개 중 5개(Eval로 description만 고쳐 트리거 개선 — Anthropic skill-creator, 2026.03), ③ -44%(스킬로 비용 44% 절감 + 완결성 Eval로 답 누락 포착 — Arize, 2026.07).
• Eval이 답하는 4가지 질문 (#36): routing(진단용, 점수 제외), uplift(none vs v1), regression(v1 vs v2), A/A(v1 vs v1b 바닥 노이즈 확인).
• pass@3 vs pass^3 (#39, Yao et al. τ-bench): 1회 정확도 80%인 에이전트를 3번 실행할 때, 적어도 1번 성공할 확률 pass@3 = 99.2%(버그 패치용)이지만, 3번 모두 성공할 확률 pass^3 = 51.2%(파급 반경 find-all용: 한 번이라도 놓치면 장애)로 급락한다.

• Find-All 두 줄 채점 (#42): 전체 4,488개 파일을 다 찍으면 Recall 1.00 / Precision 0.09가 된다. 꼼수를 막기 위해 Recall >= 0.9 AND Precision >= 0.8을 FROZEN.json에 사전 동결한다.
• "0점에도 이름이 있다" (skill_eval/checks.py, #38 & #43): ① 정답 훔쳐보기 적발 시 outcome: 'invalid', score: None(분모에서 제외 및 경보), ② 읽기 전용 워크스페이스 훼손 시 outcome: 'safety_violation', score: 0.0, ③ 정답 조건 미달 시 outcome: 'fail', score: 0.0(모두 LLM Judge 스킵). 통과한 답안만 claude-opus-5-5 Judge(항목당 3표 다수결)로 채점한다.

4. Eval 프로세스 4대 샌드박스 격리 원칙 (`skill_eval/isolation.py` · Slides #37, #44, #46)

LLM 에이전트는 목표 달성을 위해 파일 시스템과 Git 로그를 뒤지는 보상 해킹(Reward Hacking) 특성이 있다. 데이터 오염을 막고 오직 테스트 대상 스킬의 순수 기여도(Delta)만 검증하기 위해 다음 4가지를 원천 차단하며, 매 실행 전 skill-eval probe (21/21 PASS)로 무결성을 검증한다:

차단 항목	에이전트가 시도하는 반칙 (Cheating)	skill_eval/isolation.py 실제 구현 및 차단 효과
1. 원본 Repo 차단	git log/diff나 개발 문서를 읽고 출제 의도 파악	매 trial마다 $TMPDIR 아래 APFS clone(cp -c -R) 생성, git remote remove origin, .claude/.agents 제거 및 원본 스킬 경로 deny -> 실제 배포 상태 모사
2. 정답 차단	채점 스크립트나 정답 파일을 find/grep으로 베끼기	answer-keys/를 워크스페이스 외부에 격리 + permissions.deny 등록 + 접근 시도 적발 시 0점이 아닌 outcome: invalid로 즉시 실격 처리
3. ~/.claude 차단	전역 CLAUDE.md 지침, 개인 스킬, MCP, 메모리에 의존	매 trial 빈 CLAUDE_CONFIG_DIR 생성 + 번들 스킬 22종 전부 off + disableAllHooks + 상위 폴더에 CLAUDE.md 존재 시 실행 거부 -> 순수 Delta 측정
4. 네트워크 차단	막히는 부분을 구글링하거나 외부 패키지 설치로 해결	OS 커널 샌드박스(sandbox: {enabled: True, allowUnsandboxedCommands: False}) 강제 -> 시간에 구애받지 않는 결정론적 재현성(Determinism) 확보

5. Eval 성숙도 6단계 (L0~L5) 및 300-Trial 성적표 대시보드 (Slides #48, #51)

• 성숙도 6단계 (#48): L0(감/Vibe 테스트) -> L1(결정론적 체크) -> L2(고정 케이스셋 10~30개 FROZEN.json) -> L3(LLM Judge + k회 실행 + 베이스라인 대조) -> L4(PR마다 CI 자동 실행 — 가장 큰 점프) -> L5(프로덕션 실패를 새 회귀 케이스로 편입 — 가장 중요한 단계).
• 300-Trial 성적표 (#51, $65.61): none(88%, 68.3s, $0.25) vs v1(93%, 39.0s, $0.21) vs v2(95%, 38.6s, $0.21) vs v1b(93%, 40.2s). 스킬 적용으로 소요 시간 43% 단축(68.3s->38.6s) 및 비용 16% 절감. 단, 95% 신뢰구간(CI) 막대가 0을 걸치면 "차이를 확신할 수 없다"고 정직하게 판독해야 한다.

6. 모바일 통신 물리계층(PHY) SW 그룹 2nd Brain & Eval 적용 청사진

계층	second-brain-eval-kit 아키텍처	모바일 PHY / 임베디드 C/C++ SW 적용 모델
Code Graph	query.mjs: TS AST + complete/truncated/partial 라벨	ARM Clang AST 기반 MMIO 레지스터 맵, 비트필드, ISR 벡터, 3GPP FSM 심볼 링킹 + 함수 포인터 테이블 구간 partial 라벨 자동 부여
Code Memory	why.mjs: SQLite FTS5 BM25(3.0, 1.0) + RRF + 후보 1~3개 규칙	3GPP 릴리즈 변경점, 실리콘 에라타(Silicon Errata), 과거 IODT 필드 이슈를 BM25+RRF로 인덱싱하여 10us 딜레이 사유(Why) 즉시 복원
Firmware Eval	checks.py + isolation.py (4대 차단) + pass^3 기준	4대 무균 샌드박스 + invalid/safety_violation/fail 분리 채점 + 레지스터 파급 분석 시 pass^3 & Recall>=0.9/Prec>=0.8 CI 게이트 강제

7. 결론 및 엔지니어 철학: 에이전트 시대, 인간의 주도권 (Slide #55)

"에이전트를 돌리느라 24시간 쉬지 못하고 건강과 가족을 해치는 것은 도구의 본질을 전도시키는 것입니다. 에이전트에게 주도권을 뺏기지 마세요. 나의 1번 가치(가족, 건강, 엔지니어링의 본질)를 확고히 세우고, 에이전트는 나를 그 가치로 데려다주는 강력한 지렛대로 부려야 합니다." — 설계와 전략은 인간 테크리드가 쥐고, 구현과 검증은 시스템 하네스(2nd Brain + Eval CI)에 위임하라.